

FS-03: Refund Portal

Full Stack Engineer (MERN)

Time: about 3 hours (additional tasks are optional)

ID: FS-03

Scenario

Our operations team handles customer refund requests. An agent requests a refund and a supervisor approves or rejects it. Refunds must never exceed the original payment.

Must have

All amounts in the API are **integers in paise** (for example, `50000` = ₹500).

Backend (Node.js with Express or NestJS, MongoDB)

- Seed about 20 payments (id, customer, amount, status `SUCCESS` or `FAILED`) and two users: an `agent` and a `supervisor`. Identify the user with a simple header or login.
- `GET /payments`: list of payments.
- `GET /payments/:id`: payment details with its refunds.
- `POST /payments/:id/refunds` (agent only): request a full or partial refund `{ amount, reason }`. The refund starts as `PENDING`.
 - only `SUCCESS` payments can be refunded
 - approved + pending refunds must never exceed the payment amount
- `GET /refunds?status=PENDING`: refunds waiting for a decision.
- `POST /refunds/:id/approve` and `/reject` (supervisor only): moves a `PENDING` refund to `APPROVED` or `REJECTED`. A decided refund cannot be decided again.

Frontend (React or Next.js)

- Payments list.
- Payment detail page with refund history and a "Request refund" form.
- Pending refunds list with Approve / Reject buttons for the supervisor.

Additional tasks (optional, plus points)

Not required. Each one you complete counts in your favour.

- Add MongoDB indexes for your main queries and explain each one in your README.
- Audit log (who did what, when).
- Search and filters on the payments list.
- Tests for the refund limit and role rules.

Answer in your README (a few lines each)

1. Two agents request a refund on the same payment at the same moment. How do you stop the total going over the limit?
2. What would you add before this went to production?

If you run out of time, list what you skipped and how you would build it.